

VITyarthi — Computer Vision Evaluated Project

SmartDoc-Vision

Computer Vision Based Document Scanner & Image Quality Analyzer

Course: Computer Vision

Student Name: _____

University / Section: _____

Academic Project Report

1. Introduction

SmartDoc-Vision is a lightweight computer vision application that converts a photograph of a paper document into a cleaner, more readable digital image. The system detects the document boundary, estimates four corner points, corrects perspective distortion, enhances the resulting image and produces a quality report.

The project uses classical computer vision techniques so that the complete workflow can be executed locally from a terminal without requiring a trained deep-learning model or external model weights. This makes the implementation reproducible and suitable for demonstrating fundamental image-processing concepts.

2. Problem Statement

When a student photographs notes, an assignment or a form, the page may appear tilted or trapezoidal because the camera is not directly above it. Uneven illumination, low contrast, noise and blur can also reduce readability. Manual cropping and correction takes time.

The proposed system automates the common processing stages: document boundary detection, perspective correction, image enhancement and quality analysis. The output is a set of processed images plus a machine-readable JSON report.

3. Objectives

The objectives are:

1. Accept an image from the command line.
2. Detect a likely rectangular document using edges and contours.
3. Estimate four corner points using polygon approximation.
4. Correct perspective using a homography transformation.
5. Enhance the corrected image using grayscale conversion, denoising and CLAHE.
6. Generate an adaptive binary threshold image.
7. Measure brightness, contrast, blur and edge density.
8. Save all results in an organized output directory.
9. Provide validation and automated tests.

4. Functional Requirements

Module	Input	Processing	Output
1. Document Detection	Color image	Grayscale → blur → Canny → contours → polygon approximation	Four corners / not found
2. Perspective Correction	Image + four corners	Point ordering + perspective transform	Rectified document
3. Enhancement	Rectified image	Denoising + grayscale + CLAHE + adaptive threshold	Enhanced + threshold images
4. Quality Analytics	Enhanced image	Brightness, contrast, Laplacian variance, edge density	Quality metrics JSON
5. CLI / I-O	Path + options	Validation + pipeline orchestration	Terminal summary + saved files

The VITyarthi brief requires at least three major functional modules and a clear input/output workflow; this project provides five logical modules. [filecite turn0file0L24-L37](#)

5. Non-Functional Requirements

Requirement	Design approach
Performance	Resize large images during detection; process locally with OpenCV.
Usability	Simple command-line interface with clear arguments and output messages.
Reliability	Input validation, graceful no-document fallback and deterministic processing.
Maintainability	Separate Python modules for detection, geometry, enhancement, quality and I/O.
Error Handling	Explicit errors for missing/unreadable input and failed output writes.
Resource Efficiency	No database or model download; uses a small dependency set.

6. System Architecture

Layer	Components
Input / CLI	main.py, command-line arguments, image path validation
Processing	document_detector.py → perspective.py → enhancement.py → quality.py
I/O	io_utils.py, output image files and quality_report.json
Testing	tests/test_core.py

Architecture flow: User → CLI → Input Validation → Document Detection → Perspective Correction → Enhancement → Quality Analytics → Saved Images + JSON report.

7. Design Diagrams

Workflow Diagram

Start	Read image	Validate input	Detect document	Correct p erspective	Enhance	Calculate quality	Save results	End
-------	------------	----------------	-----------------	-------------------------	---------	----------------------	-----------------	-----

Component Diagram

CLI	Input/Output	Detector	Perspective	Enhancement	Quality
-----	--------------	----------	-------------	-------------	---------

Sequence Summary

User runs command	CLI calls detector	Detector returns corners	Perspective returns rectified image	Enhancer creates outputs	Quality returns metrics	CLI reports results
----------------------	-----------------------	--------------------------------	--	--------------------------------	-------------------------------	------------------------

Full Mermaid versions of the Use Case, Workflow, Component and Sequence diagrams are included in docs/diagrams.md.

8. Implementation Details

Document Detection

The image is resized for faster analysis when necessary. It is converted to grayscale, smoothed with Gaussian blur, processed with Canny edge detection and closed morphologically. External contours are sorted by area. A contour is accepted when it is sufficiently large, convex and approximates to four vertices.

Perspective Correction

The four points are ordered as top-left, top-right, bottom-right and bottom-left. A destination rectangle is created from the maximum detected width and height. OpenCV's perspective transform maps the photographed quadrilateral into this rectangle.

Image Enhancement

The rectified image is converted to grayscale, lightly denoised and enhanced using CLAHE. Adaptive Gaussian thresholding produces a binary representation that can make text boundaries easier to inspect.

Quality Analytics

Brightness is the mean grayscale intensity. Contrast is the grayscale standard deviation. Blur is approximated using the variance of the Laplacian. Edge density is the proportion of non-zero pixels after Canny edge detection. A simple heuristic labels the image GOOD, FAIR or NEEDS_REVIEW.

9. Dataset / Input Description

This project does not train a machine-learning model, so a labeled dataset is not required. The input is an ordinary document photograph. A synthetic sample image is included to make the repository immediately runnable and reproducible. For a larger evaluation, users can supply photographs of notes, forms and printed pages with different angles and lighting conditions.

10. Results

The included demonstration was executed locally. Document detection returned **found**. The sample produced brightness 226.54, contrast 41.72, blur variance 165.97, edge density 0.0295, and quality label **FAIR**.

Figure 1. Generated sample input

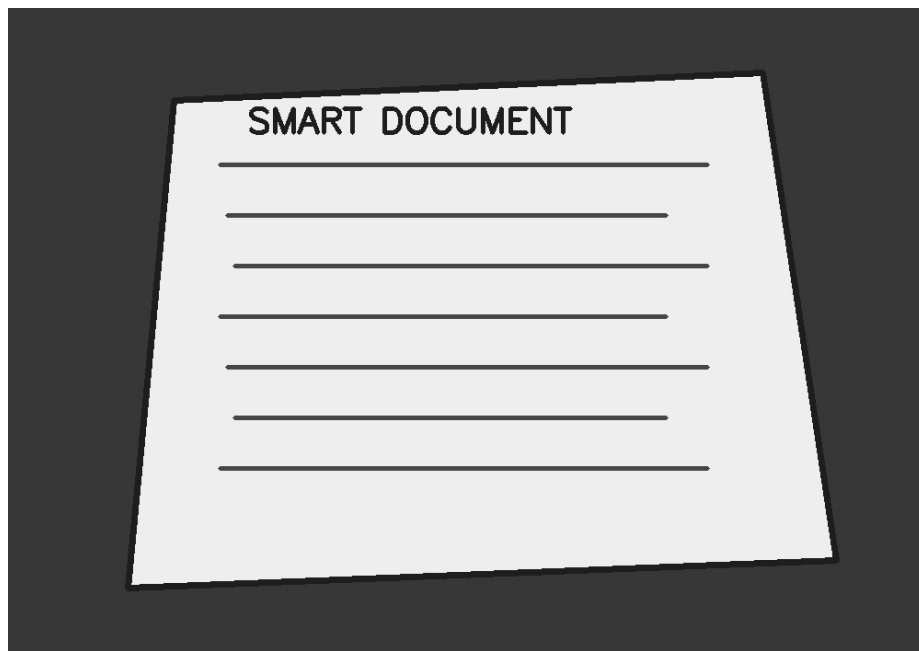


Figure 2. Detected document boundary

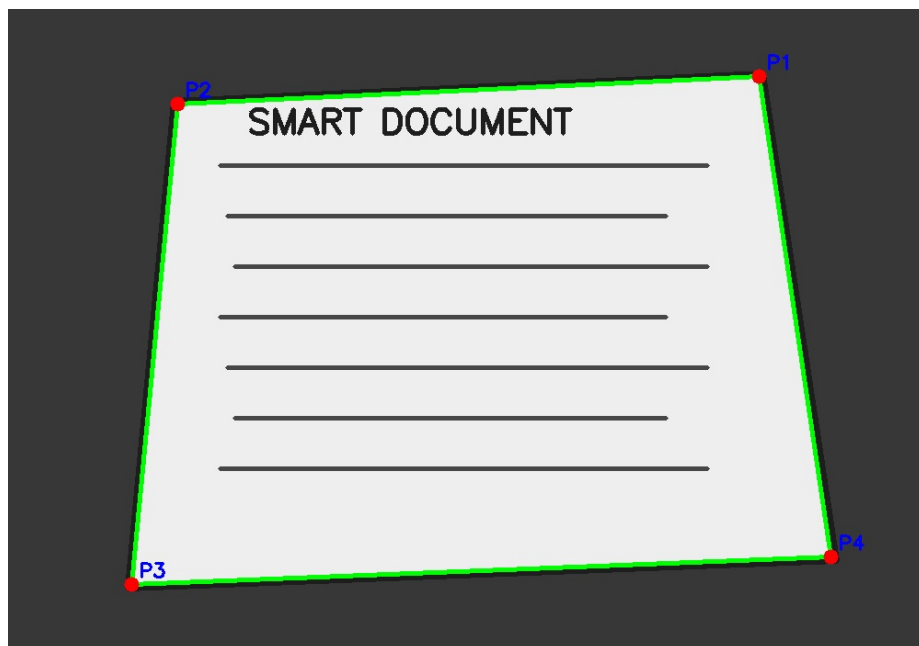


Figure 3. Perspective-corrected document

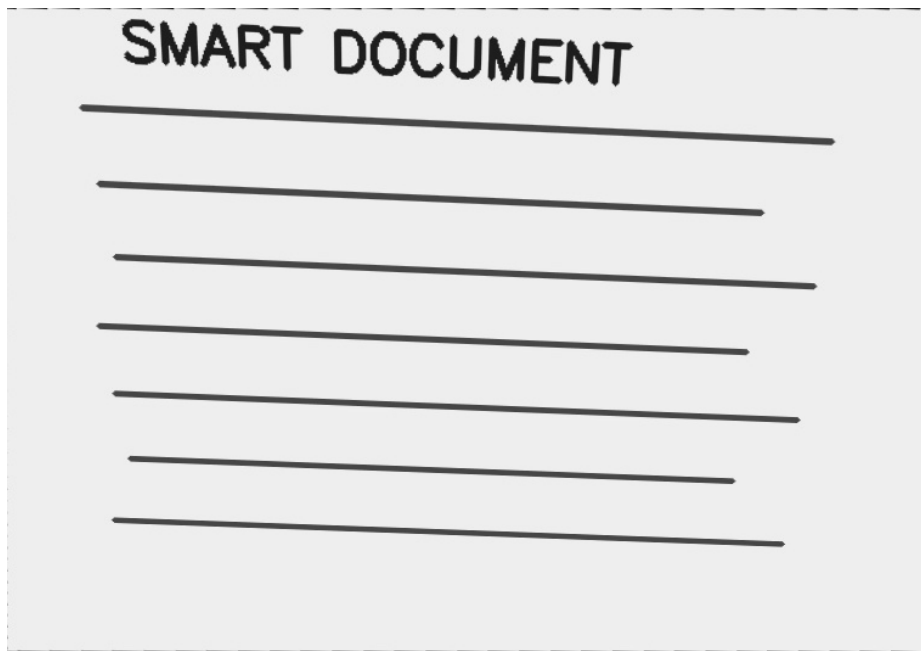
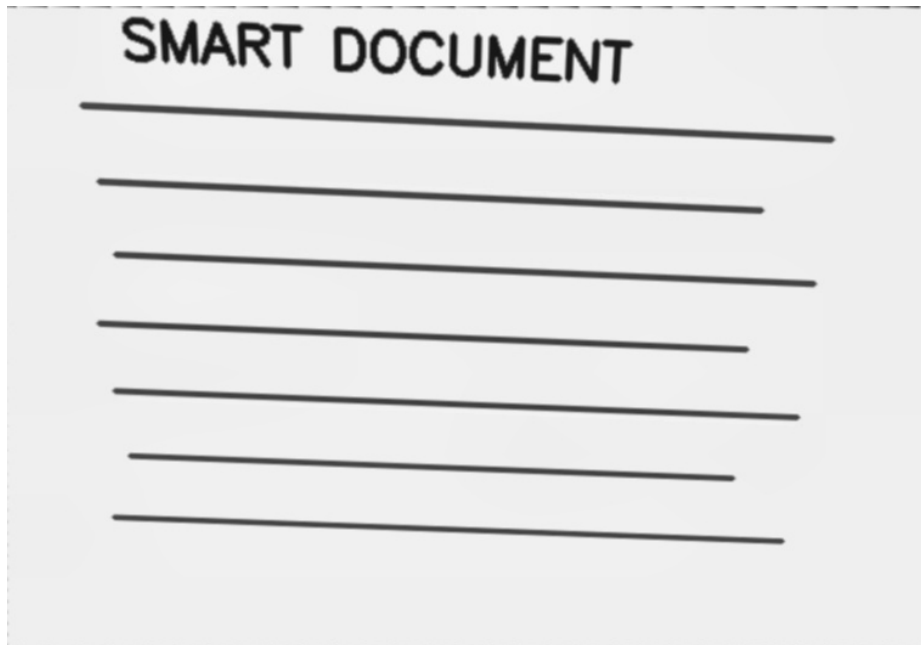


Figure 4. Enhanced grayscale document



11. Testing Approach

The repository contains unit tests using Python's unittest framework. The current test suite verifies point ordering, perspective transformation dimensions and the presence of quality metrics. The demo pipeline was also executed end-to-end from the terminal and completed successfully.

Test	Purpose	Result
test_order_points	Checks four-point geometric ordering	PASS
test_perspective_transform_shape	Checks rectification produces expected dimensions	PASS
test_quality_metrics	Checks quality-analysis output fields	PASS

The VITyarthi instructions call for testing wherever applicable and also specify modular source code and a proper folder/package structure. ■filecite■turn0file0■L51-L63■

12. Challenges Faced

- Finding the document boundary reliably when photographs contain shadows or clutter.
- Ordering the four corner points consistently before applying a perspective transform.
- Choosing quality thresholds that are understandable for an academic prototype without claiming professional scanning accuracy.
- Keeping the project executable from a terminal with a small and reproducible dependency set.

13. Design Decisions & Rationale

- **Classical CV instead of a deep model:** reduces setup complexity and avoids model-weight downloads while demonstrating core syllabus concepts.
- **File-based output:** makes results easy to inspect and avoids unnecessary database infrastructure for a single-image prototype.
- **Modular architecture:** isolates detection, geometry, enhancement and quality analysis so each component can be tested or replaced.
- **CLI first:** satisfies reproducible execution and makes evaluation independent of a GUI.

14. Learnings & Key Takeaways

- Learned how edge detection and contours can be combined to locate geometric objects.
- Understood why perspective correction is useful for document images.
- Applied homography-based image warping using four point correspondences.
- Used contrast enhancement and adaptive thresholding for document readability.
- Learned how simple numerical image statistics can support quality assessment.
- Practiced modular project organization, command-line execution and automated testing.

15. Future Enhancements

- OCR integration for searchable extracted text.
- Automatic orientation detection and page rotation.
- Multi-page scanning and PDF export.
- Deep-learning document segmentation for difficult backgrounds.
- Learned image-quality classification using a labeled dataset.
- Web or mobile interface for camera-based scanning.

16. Limitations

- The current detector expects a reasonably visible, approximately rectangular document.
- Severe occlusion, very low contrast or complex backgrounds can prevent four-corner detection.
- Quality labels are heuristic indicators rather than certified scanning measurements.
- The prototype processes one image per command.

17. Repository & Execution

The repository includes README.md, statement.md, modular source files, tests, diagrams, a sample generator and output examples. The README provides environment setup, dependency installation, execution and testing commands. The VITYarthi brief specifically requires a root-level README with setup/run/testing information and a public repository URL for submission.

■filecite■turn0file0■L87-L106■

Basic commands:

```
python -m venv .venv
pip install -r requirements.txt
python generate_sample.py
python main.py --input data/sample_input/sample_document.jpg --output data/output
python -m unittest discover -s tests -v
```

18. References

- OpenCV documentation and Python API concepts for image processing.
- NumPy documentation for numerical image-array operations.

- VITyarthi Build Your Own Project — General Project Instructions & Submission Guidelines.